



Meet people, move together.

Présentation de projet — Livrables Semaine 1

Yverdon-les-Bains · 24.08.2026

Antoine Aubry, François Bernard de Courville, Pierre Gellet, Mirco Profico

Ce que couvre cette présentation

01

Problématique & utilisateurs

Pourquoi FitMeet, pour qui

02

Solution & requirements

Objectifs, RF, RNF

03

Mockups & landing page

À quoi ressemble le produit

04

Architecture & choix techniques

KMP, Supabase, MapLibre

05

Processus de travail

Kanban, Git flow, rôles

06

Pipeline DevOps

Outils, CI/CD, démonstration

Trouver un partenaire de sport reste étonnamment fragmenté

Selon le sport et la ville, les pratiquants jonglent entre cercle d'amis, groupes WhatsApp ou Facebook, forums, clubs et applications différentes. Ces outils marchent quand on connaît déjà la communauté — mais pas quand il faut trouver vite quelqu'un qui coche plusieurs critères à la fois.

Sport

Lieu

Niveau

Disponibilité

Le problème n'est pas l'absence d'activités, c'est la difficulté à faire correspondre ces 4 critères, simplement.

Le marché confirme le besoin

- Smatch, PoteSport, Sports-Connect, SportWithYou : toutes cherchent à mettre en relation sportifs et événements.
- Strava a lancé la découverte d'événements locaux par lieu et sport.
- Marché confirmé, mais fragmenté entre apps sociales, communautés, clubs et outils spécialisés.
- Frein additionnel identifié : manque de temps et de motivation freinent la pratique physique régulière.

Trois profils, un même besoin



Lucas, 24 ans

Sportif occasionnel

BESOIN

Trouver rapidement quelqu'un de son niveau, disponible près de chez lui.

FRUSTRATION

Vient d'arriver en ville, ne veut pas rejoindre 3 groupes WhatsApp pour organiser une séance.



Sophie, 32 ans

Sportive régulière

BESOIN

Une activité qui colle précisément à son emploi du temps, son niveau, sa localisation.

FRUSTRATION

Ses partenaires habituels ne sont pas toujours dispo ; organiser prend plusieurs messages.



Marc, 40 ans

Responsable de club (volley)

BESOIN

Rendre ses entraînements ouverts visibles et remplir les places disponibles.

FRUSTRATION

Infos dispersées entre site du club, Instagram, WhatsApp et bouche-à-oreille.

Problem statement

Les personnes souhaitant pratiquer un sport à plusieurs rencontrent aujourd'hui des difficultés à trouver, au bon moment et près de chez elles, des partenaires ou événements correspondant à leur sport, leur niveau et leurs disponibilités. Les solutions existantes sont dispersées entre réseaux sociaux, groupes privés, clubs et applications spécialisées.

FitMeet centralise cette découverte pour rendre l'organisation et la pratique sportive locale plus simples, accessibles et spontanées.

Élément de validation marché

Smatch indique déjà plusieurs milliers de partenaires potentiels à Lausanne, et plusieurs centaines à Genève ou Zurich — preuve d'une demande locale réelle et importante.

FitMeet : centraliser la découverte sportive locale

Particuliers, clubs et associations créent des activités (sport, heure, date, lieu, niveau, durée). Les autres utilisateurs les découvrent sur une carte interactive ou une liste filtrable, puis les rejoignent.

1

Faciliter la pratique collective

Trouver un partenaire ou groupe, quel que soit le niveau ou la disponibilité.

2

Fédérer une communauté locale

Connecter particuliers, clubs et associations d'un même bassin géographique.

3

Donner de la visibilité aux clubs

Un canal simple pour promouvoir les séances et remplir les créneaux.

4

Simplifier l'organisation

Réduire la friction : plus de groupes WhatsApp ou affichages papier.

5

Encourager la régularité

Découverte facile d'événements récurrents et de pratiquants similaires.

6

Garantir une découverte fluide

Carte interactive + liste filtrable + recommandations pertinentes.

Requirements fonctionnels

COMPTES UTILISATEURS

RF-01 · Créer un compte (particulier / club) et s'authentifier

RF-02 · Compléter son profil (photo, sports, niveau, localisation)

RECHERCHE & DÉCOUVERTE

RF-08 · Visualiser les événements sur une carte interactive

RF-09 · Visualiser les événements sous forme de liste

RF-10 · Filtrer (sport, date, niveau, distance, places)

RF-11 · Consulter le détail avant de rejoindre

GESTION DES ÉVÉNEMENTS

RF-03 · Créer un événement (sport, date, lieu, niveau, places, prix)

RF-04 · Modifier ou annuler un événement créé

RF-05 · Rejoindre un événement, dans la limite des places

RF-06 · Se désinscrire d'un événement

RF-07 · Consulter la liste des participants (créateur)

NOTIFICATIONS

RF-12 · Être notifié d'un changement sur un événement rejoint

Envisagé mais non confirmé pour la v1 : messagerie entre participants, avis/notation, suivi d'organisateur.

Requirements non-fonctionnels

Performance

Recherches et affichage carte fluides : < 2 secondes en usage normal.

Fiabilité

Service disponible en quasi continu, avec sauvegarde régulière des données.

Sécurité & confidentialité

Conformité RGPD, mots de passe sécurisés, échanges chiffrés (HTTPS).

Utilisabilité

Interface simple et intuitive, accessible aux non-technophiles.

Compatibilité

Fonctionne sur iOS et Android, s'adapte à toutes tailles d'écran.

Maintenabilité

Code structuré de manière modulaire pour faciliter l'évolution.

À quoi ressemble FitMeet

Maquettes Figma — parcours principal de l'utilisateur



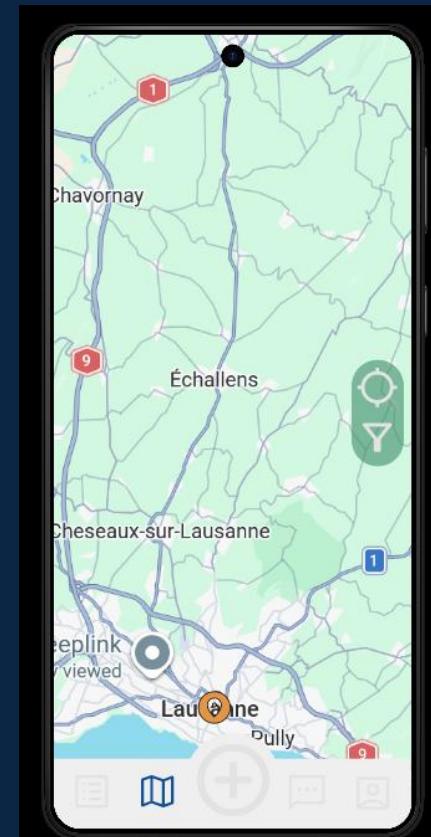
1. Connexion

Email ou Google



2. Liste & filtres

Par sport, niveau, distance



3. Carte interactive

MapLibre + OpenStreetMap



4. Chat d'événement

Coordination temps réel

De la découverte à la participation



1

Filtrer par sport

Chips Football, Basketball, Tennis en haut de la liste — un tap pour affiner.

2

Comparer les créneaux

Sport, horaire, lieu, niveau et places restantes visibles d'un coup d'œil (ex. 3/10).

3

Rejoindre en un geste

Bouton « Rejoindre » disponible tant qu'il reste de la place — RF-05.

4

Basculer vers la carte

Icône carte en barre inférieure pour une vue géographique des mêmes événements.

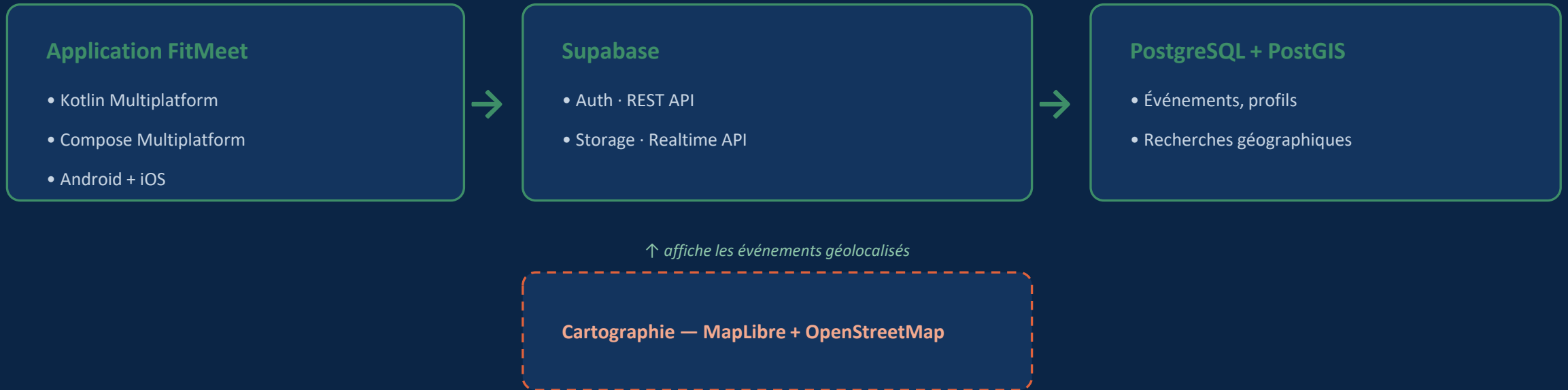
Une vitrine web pour présenter FitMeet

En parallèle de l'application mobile, une landing page présente le projet au grand public : proposition de valeur, fonctionnement en 3 étapes, et sports/cas d'usage couverts.



Architecture préliminaire

Une architecture simple, adaptée à un MVP mobile



Flux de données

Ouverture de la carte → l'app interroge Supabase → requête spatiale PostGIS sur PostgreSQL → événements retournés → affichage MapLibre. Création d'événement : même chemin, inversé.

Pourquoi Kotlin Multiplatform ?

Compose Multiplatform permet de partager l'interface utilisateur entre Android et iOS, tout en gardant un accès aux API natives lorsqu'une fonctionnalité l'exige.

Deux apps natives

Android + iOS séparés

- Meilleur contrôle plateforme par plateforme
- Code dupliqué, maintenance x2

Flutter / React Native

Frameworks cross-platform établis

- Écosystèmes matures
- Écarté : priorité à la montée en compétence Kotlin

✓ RETENU

Kotlin Multiplatform

Notre choix

- Kotlin partagé : logique + UI (Compose)
- Accès natif si nécessaire
- Anticipe le cours Android du S5

Backend & cartographie : réduire la complexité

Pourquoi Supabase ?

FitMeet a besoin d'une base de données, d'authentification, de stockage et de fonctionnalités géographiques. Supabase fournit PostgreSQL, Auth, Storage et PostGIS directement — sans construire un backend maison.

Compromis assumé

Moins de contrôle qu'un backend sur-mesure, mais les données restent basées sur PostgreSQL — pas de verrouillage propriétaire fort.

Pourquoi MapLibre + OSM ?

MapLibre est un moteur cartographique open source avec SDK natifs Android/iOS, utilisé avec les données OpenStreetMap plutôt qu'un fournisseur propriétaire.

Bénéfices

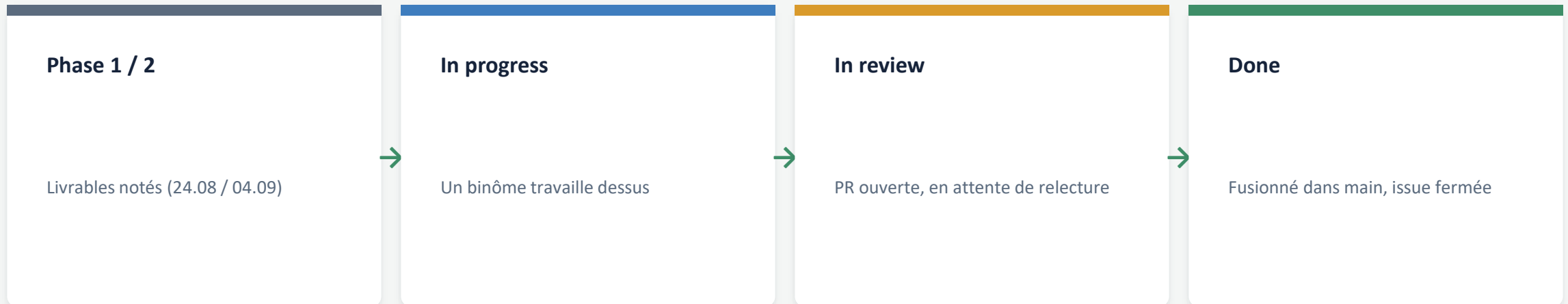
- Plus de liberté sur l'affichage et la source des cartes
- Moins de dépendance à un fournisseur unique

⚠ RISQUE IDENTIFIÉ

MapLibre Compose (wrapper Compose Multiplatform) est plus récent que les SDK natifs Android/iOS. Certaines fonctionnalités pourraient nécessiter du code spécifique par plateforme — isolé pour permettre un retour aux SDK natifs si besoin.

Kanban — GitHub Projects

Cartes rangées par date de rendu (pas de backlog unique) : chaque carte avance de Phase N → In progress → In review → Done.



Définition de « terminé »

- ✓ Toutes les cases de l'issue sont cochées
- ✓ Le build local passe (assembleDebug)
- ✓ PR relue et approuvée par quelqu'un d'autre
- ✓ Fusionnée dans main → issue fermée automatiquement

Labels & jalons

Phases 01-product → 14-presentation, priorité P1 (24.08) / P2 (04.09).

Charge de travail : pas de limite de cartes par personne — 4 personnes sur 3 semaines, chacun son domaine, protégé par le binôme sur chaque issue et le point quotidien.

Git Flow & Pull Requests

GitHub Flow : une branche principale, une branche courte par issue



Branche créée depuis l'issue (bouton « Create a branch ») : elle est liée automatiquement, et la fusion ferme l'issue + déplace la carte en Done.

main est protégée — ruleset « protect-main »

pull_request

Push direct refusé — 1 approbation requise

non_fast_forward

Force push refusé

deletion

main ne peut pas être supprimée

Qui relit : n'importe qui sauf l'auteur (idéalement le binôme de l'issue) · 1 approbation · squash ou merge · relecture le jour même

Bloque une fusion : approbation manquante, commentaire non résolu, ou build rouge une fois le pipeline en place.

Responsabilités par phase

Un responsable par phase — jamais seul : au moins deux personnes par issue, la répartition n'est pas figée.

01	Produit & exigences	@calystoxi	08	Événements	@calystoxi
02	Processus de projet	@mircoprofico	09	Carte & géolocalisation	@calystoxi
03	Design UX / UI	@lbuprofenLover	10	Recherche & filtres	@calystoxi
04	Landing page	@lbuprofenLover	11	Tests	@lbuprofenLover
05	Supabase	@fr2c	12	DevOps / CI-CD	@mircoprofico
06	Fondations app mobile	@mircoprofico	13	Documentation	@fr2c
07	Authentification & profils	@fr2c	14	Présentation	Tous

Rythme quotidien : un point par jour — ce qui a avancé, ce qui bloque, ce que chacun prend ensuite.

Outils de développement mis en place

1

GitHub Issues

ISSUE TRACKER

35 issues ouvertes, une par tâche, labellisées par phase et priorité (P1/P2).

2

GitHub Projects

KANBAN

Tableau de suivi Phase 1/2 → In progress → In review → Done.

3

GitHub Actions

CI/CD

Workflows déclenchés sur push / pull request vers main.

4

Rulesets GitHub

QUALITÉ DU CODE

Protection de main : PR + 1 approbation obligatoires avant fusion.

Dépôt du projet

github.com/mircoprofico/FitMeet

Structure Gradle multi-module : androidApp · iosApp · shared

Environnement de déploiement & pipeline CI/CD

GitHub Actions — build et vérifications automatiques à chaque changement



Environnement de déploiement

Backend hébergé sur Supabase Cloud (Auth, PostgreSQL/PostGIS, Storage, Realtime) — pas d'infrastructure serveur à maintenir. Les runners GitHub Actions construisent l'APK Android à chaque changement.

Où en est le pipeline aujourd'hui

Workflow GitHub Actions en place pour le build automatique. Le statut de build comme condition de fusion sur main est en cours de finalisation (issue #86) avant intégration complète dans les rulesets.

Démonstration du déploiement d'une modification

Ce que nous montrons en direct :

1

Créer une issue

Choisir une carte en Phase 1, l'assigner à un binôme.

2

Ouvrir une branche

Depuis l'issue via « Create a branch » — liaison automatique.

3

Commit + push

Modification poussée sur la branche courte.

4

Pull Request

Ouverture de la PR, déclenchement du build GitHub Actions.

5

Relecture & approbation

Un binôme relit ; main reste protégée entretemps.

6

Fusion dans main

Squash ou merge — l'issue se ferme automatiquement, la carte passe en Done.

Vers les livrables finaux du 04.09.2026

1

Fondations app mobile

Structure KMP, navigation, thème Compose partagé.

2

Authentification & profils

Intégration Supabase Auth, formulaires profil.

3

Événements & carte

CRUD événements, intégration MapLibre + PostGIS.

4

Tests & CI/CD complet

Tests automatisés, statut de build bloquant sur main.

Jalon : Livrables finaux — vendredi 04.09.2026



Merci — questions ?

github.com/mircoprofico/FitMeet

@calystoxi · @mircoprofico · @IbuprofenLover · @fr2c